

UI自动化测试

web自动化



黑马程序员
www.itheima.com

传智教育旗下
高端IT教育品牌

01

UI自动化介绍



黑马程序员
www.itheima.com

传智教育旗下
高端IT教育品牌

UI自动化测试介绍

认识UI自动化测试

UI自动化测试



- **UI:** User Interface, 简称UI, 用户界面是系统和用户之间进行交互和信息交换的媒介
- **UI自动化测试:** 使用程序、脚本对系统界面体现的功能和数据信息展示等进行的测试技术

Web自动化测试

移动自动化测试



- 大量的版本需要回归测试
- 手工回归测试效率比较低
- 测试工程师价值得不到体现



- **体现价值:** 测试人员需要从重复的回归测试抽离出来投入更重要功能模块测试中
- **工作效率:** 提高测试效率

UI自动化测试介绍

实施UI自动化测试前置条件



项目需要实施UI自动化测试需要满足以下前置条件：

- 项目上线发布频率高，回归测试任务重
- 项目需要实现自动化的功能模块，需求变更不频繁
 - 一般只实现核心功能模块
- 项目周期要长
 - 公司自研或者公司核心产品

UI自动化测试介绍

UI自动化测试执行时机



- 项目UI界面还没有，能编写脚本吗？
- 项目刚提测时，能执行吗？？



不能！！！！

- 无UI无法确认UI界面信息，脚本无法编写
- 功能不稳定，问题多，UI界面可能发生变化
- 无法执行脚本调试

项目实施UI自动化测试的时机：

- 一般情况下在手工测试完成之后**
- ✍ 版本或项目功能趋于稳定

UI自动化测试介绍

UI自动化测试核心作用和劣势



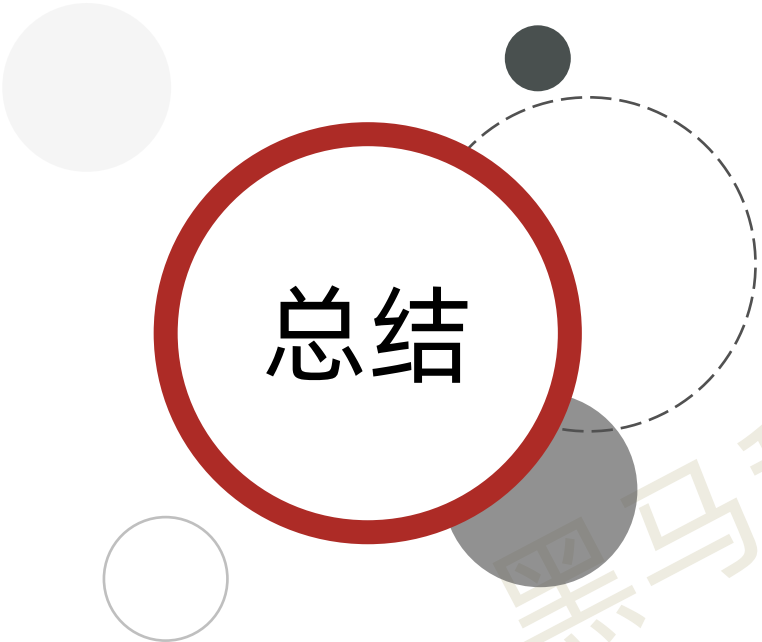
项目实施UI自动化测试之后：

- **节省人力成本：** 回归测试工作由脚本代替人去执行
- **提高回归测试效率：** 脚本执行测试速度更快
- **提高测试质量：** 一旦自动化脚本库完善后测试执行过程更标准和准确



UI自动化测试劣势：

- 对测试人员要求提高
- 前期投入成本大
- 对项目要求高



总结

1. UI自动化测试概念

- ✓ 使用程序、脚本对系统界面体现的功能和数据信息展示等进行的测试技术

2. 实施UI自动化测试前置条件

- ✓ 项目上线发布频率高，回归测试任务重
- ✓ 项目需要实现自动化的功能模块，需求变更不频繁
- ✓ 项目周期要长

3. UI自动化测试执行时机

- ✓ 一般情况下在手工测试完成之后
- ✓ 版本或项目功能趋于稳定

4. UI自动化测试的核心作用

- ✓ 节省人力成本
- ✓ 提高回归测试效率
- ✓ 提高测试质量

Web自动化测试



黑马程序员
www.itheima.com

传智教育旗下
高端IT教育品牌

01

Selenium介绍

- 认识自动化测试工具
- 了解Selenium特点
- 知道Selenium核心组件
- 掌握Selenium-WebDriver原理

Web自动化测试工具介绍

常见的用于Web自动化测试的工具:



QTP是一个商业化的功能测试工具，收费，支持web，桌面自动化测试。



Robot Framework是一个基于Python具有可扩展性、支持关键字驱动测试的测试自动化框架。



Selenium是一个开源的web自动化测试工具，免费，主要做功能测试。

```
*** Keywords ***
1  Connect to Server
2     Connect    fe00::aede:48ff:fe00:1122
10
11  Close Server Connection
12     Disconnect
13
14  Login User
15     [Arguments]    ${login}    ${password}
16     Set Login Name    ${login}
17     Set Password    ${password}
18     Execute Login
19
20  Verify Valid Login
21     [Arguments]    ${exp_full_name}
22     ${version}=    Get Server Version
23     Should Not Be Empty    ${version}
24     ${name}=    Get User Name
25     Should Be Equal    ${name}    ${exp_full_name}
```

```
*** Test Cases ***
Run Test
Login User with Password
    Connect to Server
    Login User    ironman    1234567890
    Verify Valid Login    Tony Stark
    [Teardown]    Close Server Connection

Run Test
Denied Login with Wrong Password
    [Tags]    negative
    Connect to Server
    Run Keyword And Expect Error    *Invalid Pas
    Verify Unauthorised Access
    [Teardown]    Close Server Connection
```

Selenium特点



市场工具多样，如何选择?



Selenium 特点:

- **基于web项目回归测试角度考虑:**

- ✎ 项目的兼容性：平台、浏览器

- **基于脚本编写和维护角度考虑:**

- ✎ 编程语言选择
- ✎ 工具本身使用成本



- 开源软件
- 跨平台
- 支持多种浏览器
- 支持多种语言
- 功能强大、成熟稳定

Selenium核心组件

Selenium

Selenium-IDE

浏览器插件，录制脚本工具。支持脚本回放和导出。
可以导出Python和Java等类型的单元测试脚本



Selenium-Grid

分布式执行自动化测试用例工具，用于大批量测试用例的执行

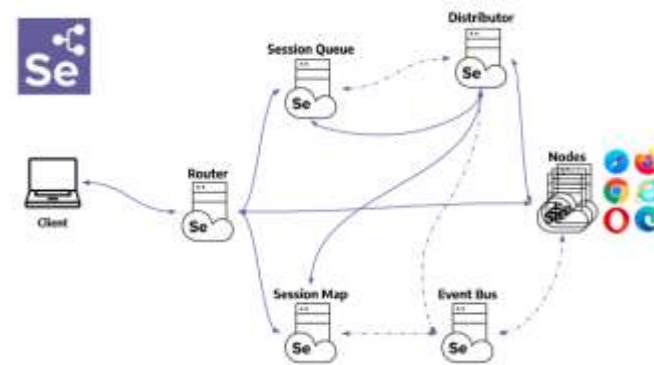


Selenium-Webdriver

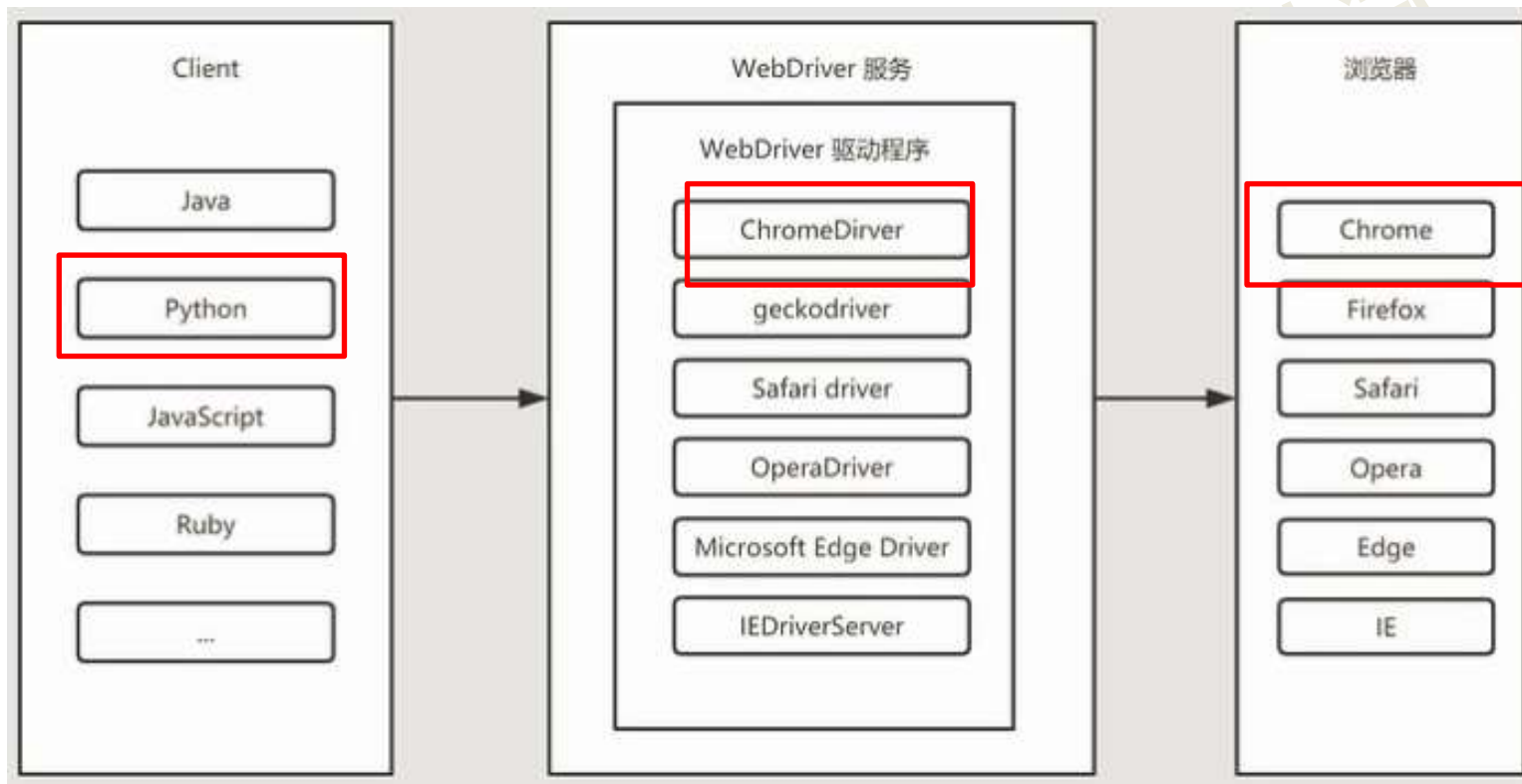
脚本编写核心工具，提供模拟手工操作的常用方法



https://www.itcast.cn	
Command	Target
1 open	/
2 set window size	784x728
3 mouse over	css=li:nth-child(1) > .li_in img
4 mouse out	css=li:nth-child(1) > .li_in img
5 click	linkText=关于传智
6 store window handle	root
7 select window	handle=\${win8443}



Selenium-WebDriver原理



总结

1. 自动化测试工具

- ① QTP
- ② Robot Framework
- ③ Selenium

2. Selenium特点

- ①开源，跨平台
- ②支持多浏览器，多种语言
- ③功能强大，技术成熟稳定

3. Selenium核心组件

- ①Selenium-IDE
- ②Selenium-Grid
- ③Selenium-Webdriver

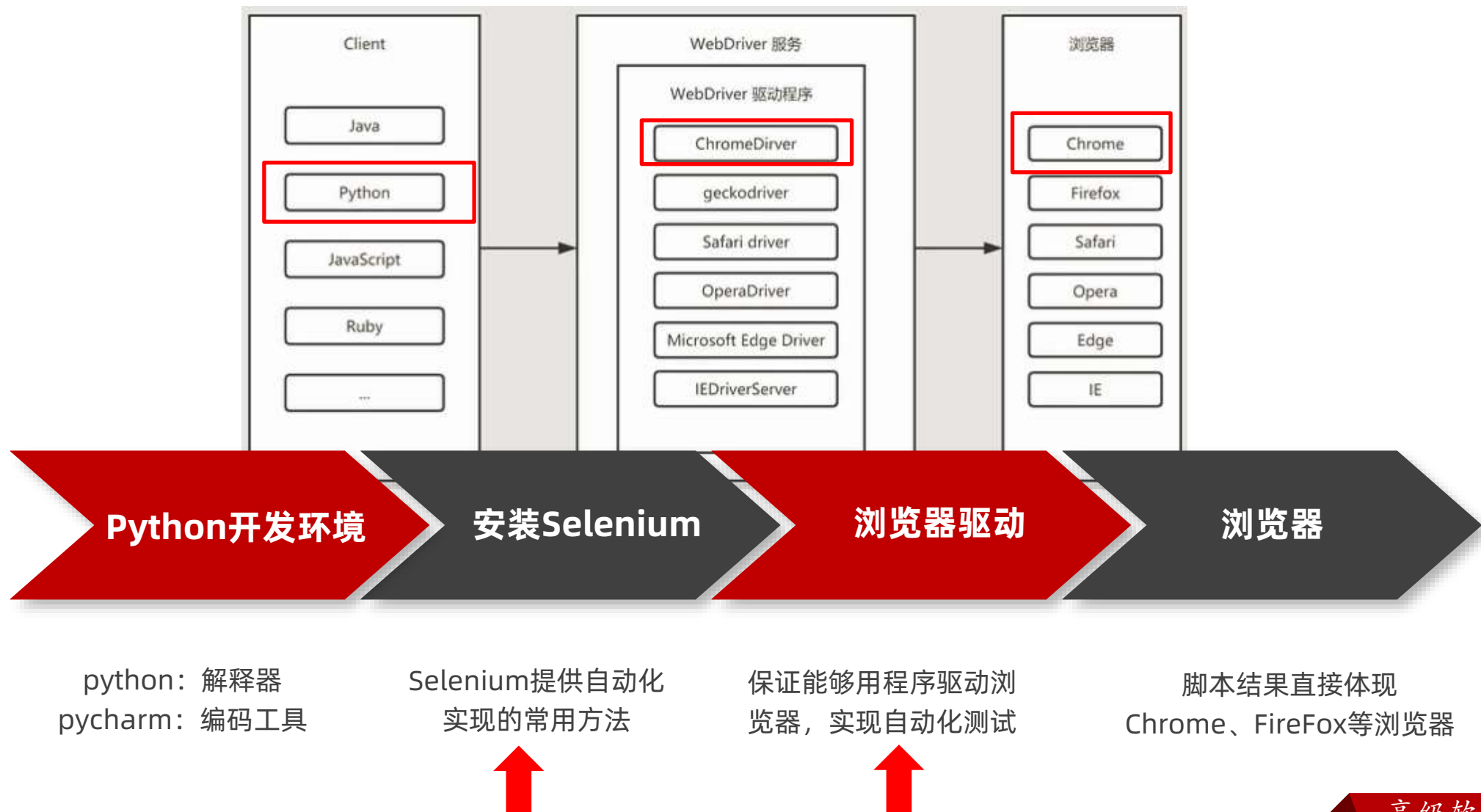
4. Selenium-Webdriver原理

Python代码 → Webdriver（浏览器驱动程序） → 浏览器

02

Selenium环境搭建

Web自动化测试环境搭建步骤



Web自动化测试环境搭建步骤

- 理论步骤是4步
 - ① 搭建Python开发环境
 - ② 安装Selenium库
 - ③ 下载安装浏览器驱动
 - ④ 下载安装浏览器
- 实际操作步骤3步
 - ① 安装Selenim库
 - ② 下载安装浏览器驱动
 - ③ 检查Web自动化测试环境

安装Selenium



安装包

```
pip install selenium
```



查看包

```
pip show selenium
```



卸载包

```
pip uninstall selenium
```

浏览器驱动



- 1、下载浏览器驱动程序:

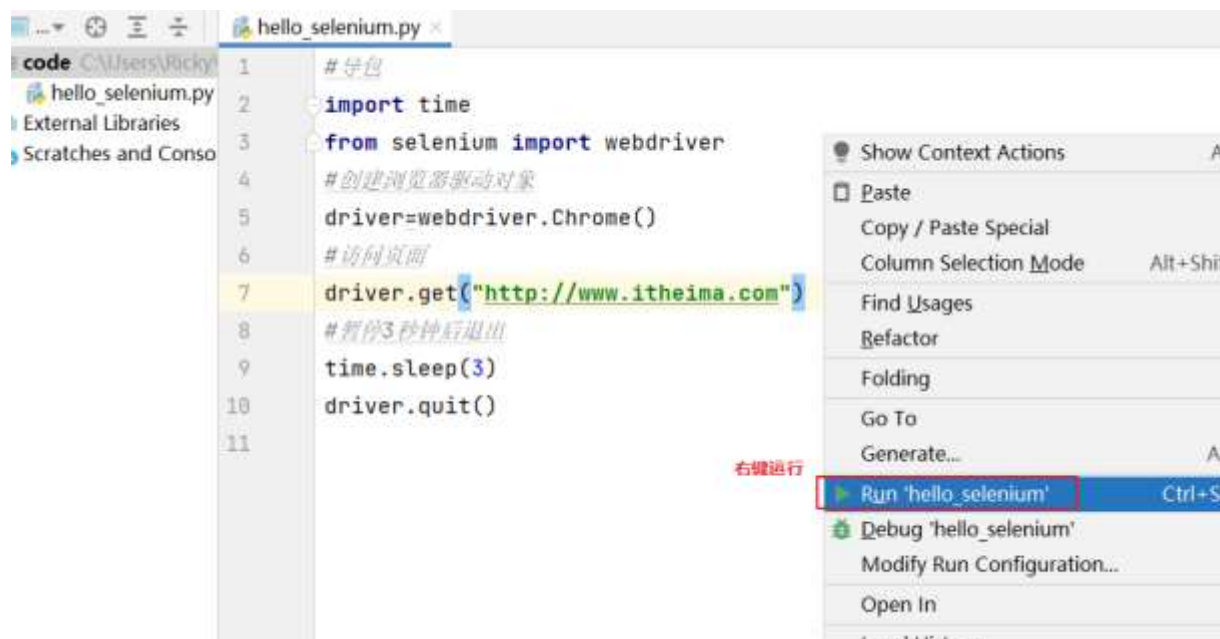
- ✎ 不同浏览器都有自己独立驱动程序
- ✎ 不同的版本需要下载匹配其版本的驱动程序
- ✎ 驱动下载链接: https://www.selenium.dev/documentation/webdriver/getting_started/install_drivers/#quick-reference

- 2、配置驱动文件:

- ✎ 可直接将驱动文件拷贝到python的安装目录

案例 检测环境代码

运行已经给大家的python文件



```
1 # 导包
2 import time
3 from selenium import webdriver
4 # 创建浏览器驱动对象
5 driver=webdriver.Chrome()
6 # 访问页面
7 driver.get("http://www.itheima.com")
8 # 等待3秒钟后退出
9 time.sleep(3)
10 driver.quit()
11
```

右键运行

运行结果



总结

1. Web自动化测试环境搭建步骤

- ①Python开发环境
- ②安装Selenium
- ③下载浏览器驱动
- ④浏览器

2. 浏览器驱动下载注意事项

- ①不同浏览器有自己独立驱动程序
- ②不同版本浏览器需要下载匹配版本的驱动程序

3. 浏览器驱动存放位置

- ①将驱动文件拷贝到python的安装目录

03

Selenium入门案例

- 掌握Selenium基本操作步骤

Selenium操作步骤

- 1.导包
- 2.创建浏览器驱动对象
- 3.访问Web页面
- 4.暂停几秒钟
- 5.浏览器驱动对象退出

黑马程序员·软件测试

案例 入门案例

通过程序启动浏览器，并打开黑马程序员首页，暂停3秒后，关闭浏览器

1.导包

```
import time
```

```
from selenium import webdriver
```

2.创建浏览器驱动对象

```
# Firefox浏览器: ff = webdriver.Firefox()
```

```
# Chrome浏览器:
```

```
driver = webdriver.Chrome()
```

3.打开Web页面

```
driver.get("https://www.itheima.com/")
```

4.暂停几秒钟

```
time.sleep(3)
```

5.驱动对象退出

```
driver.quit()
```



总结

1. Selenium基本操作步骤

- ①导包
- ②创建浏览器驱动对象
- ③访问页面
- ④暂停几秒钟
- ⑤浏览器驱动对象退出

04

元素定位工具

- 了解HTML标签数据的组成部分
- 掌握元素定位工具的基本使用
- 认识元素定位的方式

HTML介绍

Web前端三大核心技术

- HTML:负责网页的架构
- CSS:负责网页的样式、美化
- JS:负责网页的行为



什么是HTML

HTML 是用来描述网页的一种语言。



Html标签

- 单标签

- 双标签<body> 内容</body>



标签属性

- 属性格式：属性名="属性值"
京东



元素定位工具

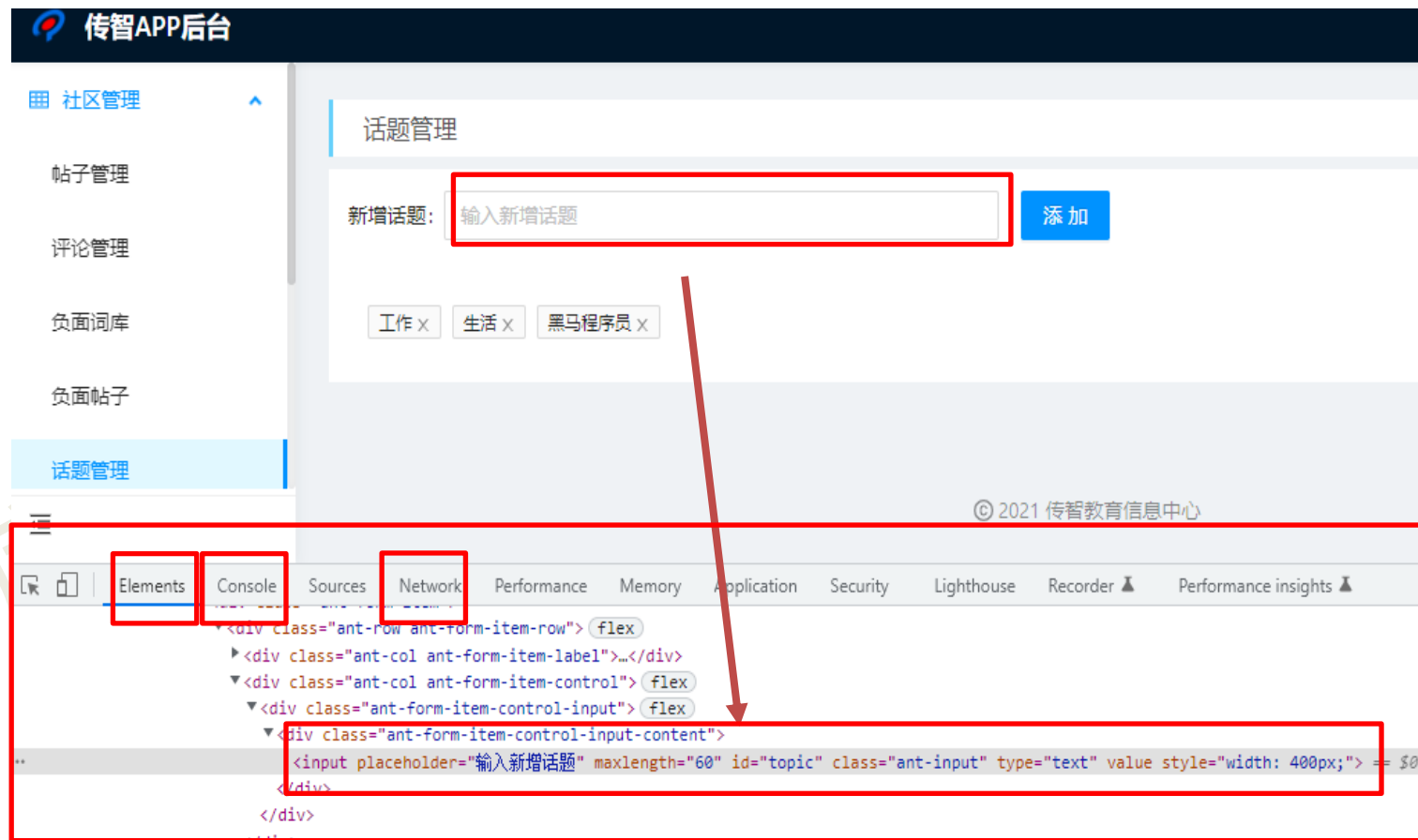


浏览器开发者工具就是给专业的web应用和网站开发人员使用的工具,包含了对HTML查看和编辑、Javascript控制台、网络状况监视等功能。

安装：无需安装，浏览器自带

作用：快速定位元素，查看元素信息

展示：在浏览器页面中点击鼠标右键，然后点击 检查



为什么要进行元素定位

账户登录



[忘记密码?](#)

 支付宝 |  微信 |  QQ

[立即注册](#)



手工测试:

通过肉眼找到输入框，再进行操作



UI自动化:

我也要操作，通过什么来找呢?

```
<input type="text" placeholder="手机号/邮箱" id="mobile" autocomplete="off">
<input type="password" placeholder="密码" id="pwd" autocomplete="off">
<input type="text" placeholder="验证码" id="code" autocomplete="off">
```

UI界面本质上就是HTML直接体现,脚本通过HTML标签信息来找到具体的元素

如何进行元素定位呢?



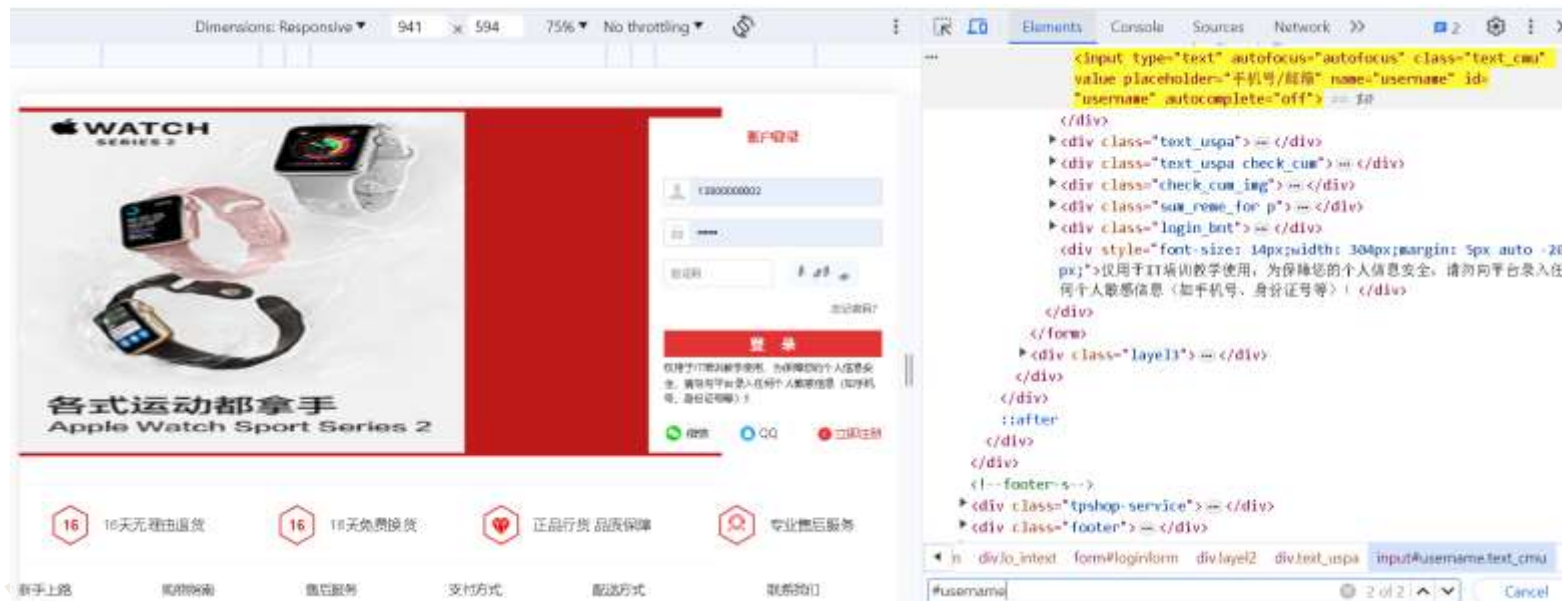
```
<input type="text" placeholder="手机号/邮箱" id="mobile" autocomplete="off">  
<input type="password" placeholder="密码" id="pwd" autocomplete="off">  
<input type="text" placeholder="验证码" id="code" autocomplete="off">
```

认识元素定位方式

定位方式	类型	说明	举例说明
id	利用元素属性	有id优先使用id	<code><input type= "text" id= "name" /></code>
name		无id可以查看是否有name	<code><input type= "text" name= "pwd" /></code>
class_name		根据class的属性值定位	<code><input class= "top" /></code>
tag_name	标签名	利用HTML标签名	<code><input /></code>
link_text	超链接定位	全部文本信息	<code><a>黑马程序员</code>
partial_link_text		局部文本信息	<code><a>学IT来黑马程序员, IT黑马</code>
CSS ★	选择器定位	id选择器 class选择器 元素选择器 属性选择器 层级选择器 属性包含	<code><div id= "main" > <input type= "text" /> 请输入用户名 </div></code>
XPath ★	路径定位	路径（相对路径） 属性 属性与层级结合 属性包含 利用元素文本	<code><div id= "main" > <input type= "text" /> 请输入用户名 </div></code>

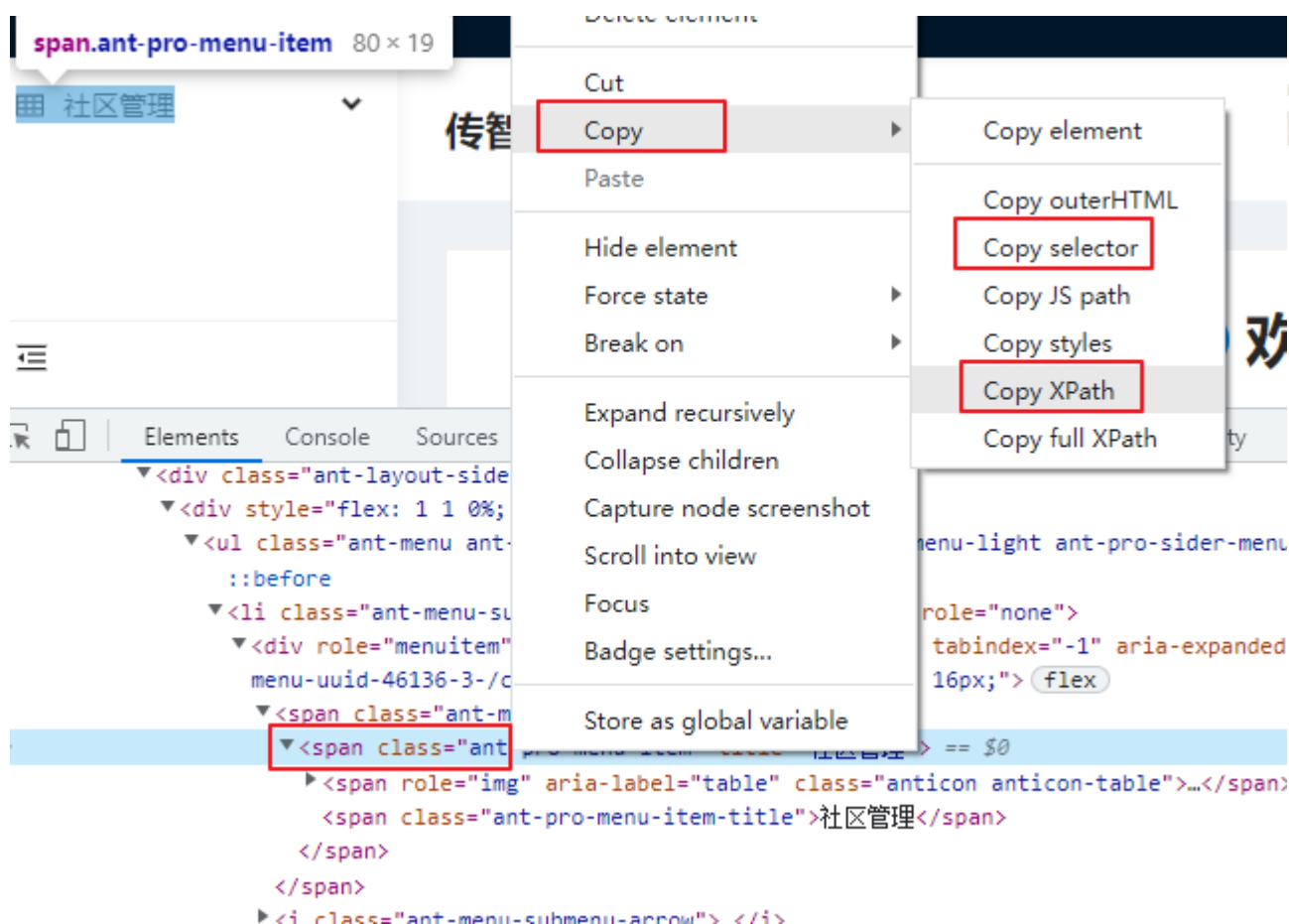
元素定位方式演示

- 根据 id值 进行元素定位
- 根据 class name值 进行元素定位
- 根据 标签名 进行元素定位
- 根据 XPath 路径 进行元素定位
- 根据 XPath属性 进行元素定位
- 根据 CSS属性 进行元素定位



小技巧

- 针对于元素定位我们可以通过浏览器的右键，复制CSS、XPath定位方式



Xpath定位-路径定位

绝对路径

- **概念：**从最外层元素到指定元素之间所有经过的元素层级的路径。
- **表达式写法：**绝对路径以 **/html根节点** 开始，使用 **/** 来分隔元素层级。

```
/html/body/div/fieldset/p[1]/input
```

相对路径

- **概念：**从目标定位元素的任意层级的上级元素开始到目标元素所经过的层级的路径。
- **表达式写法：**以 **//** 开始，后续每个层级都使用 **/** 来分隔。

```
//fieldset/p[1]/input
```

路径+属性

```
//*[ @属性名称='属性值']
```

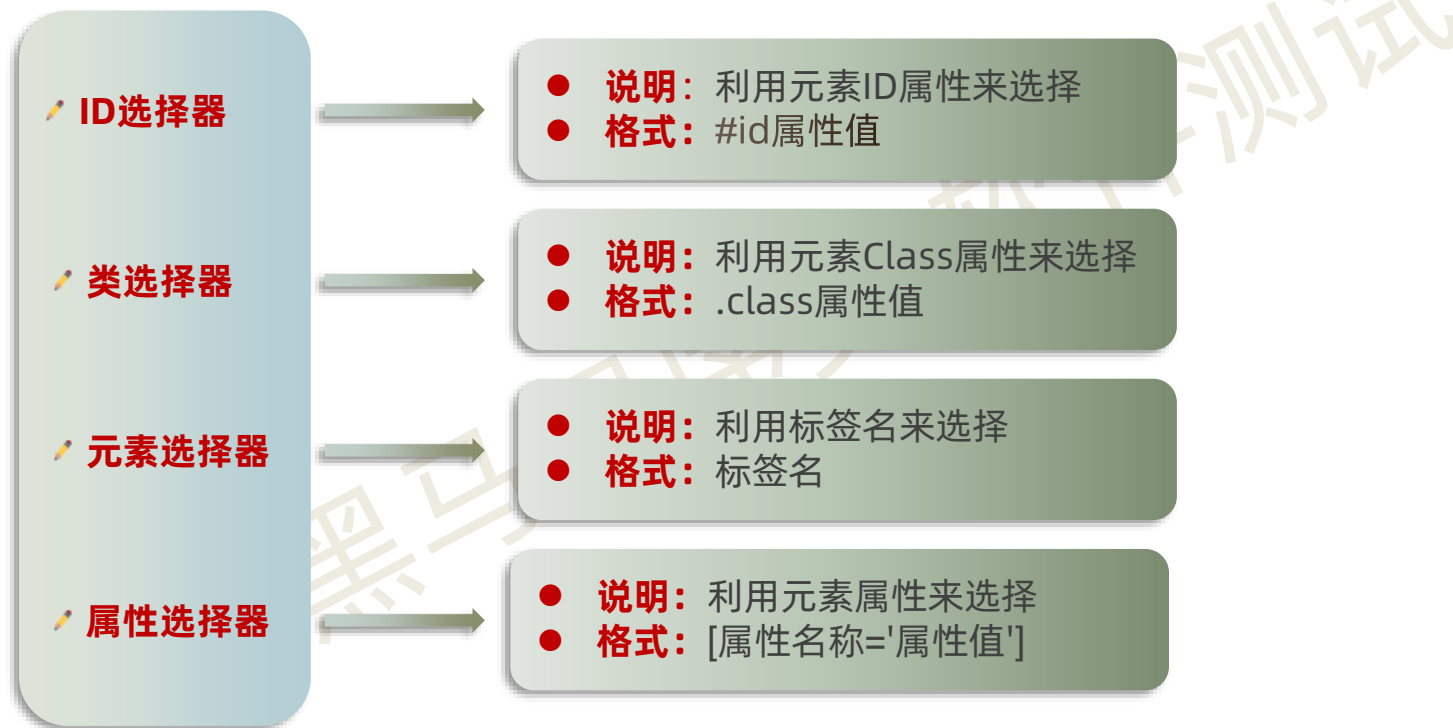
路径+文本

```
//*[text()='文本信息']
```

```
<body>
<a href="/Home/user/reg.html">注册</a>
<a class="title-content c-link c-font-medium c-line-clamp1"
href="https://www.baidu.com/s?cl=3&tn=baidutop10&fr=top1000&
wd=%E0%B0%88%E0%A1%A7%B0%E5%90%B4%E5%AD%A9%E8%BE%E6%89%8B%E6%9C%AF%E0%88%90%E5%8A%9F&
rsv_idx=2&rsv_dl=fyb_n_homepage&hisfilter=1"
target="_blank">
<span class="title-content-index c-index-single c-index-single-hot4">4</span>
<span class="title-content-title">消息称吴孟达手术成功</span>
<span class="title-content-mark c-text c-gap-left-small"></span>
</a>
</body>
```

CSS定位

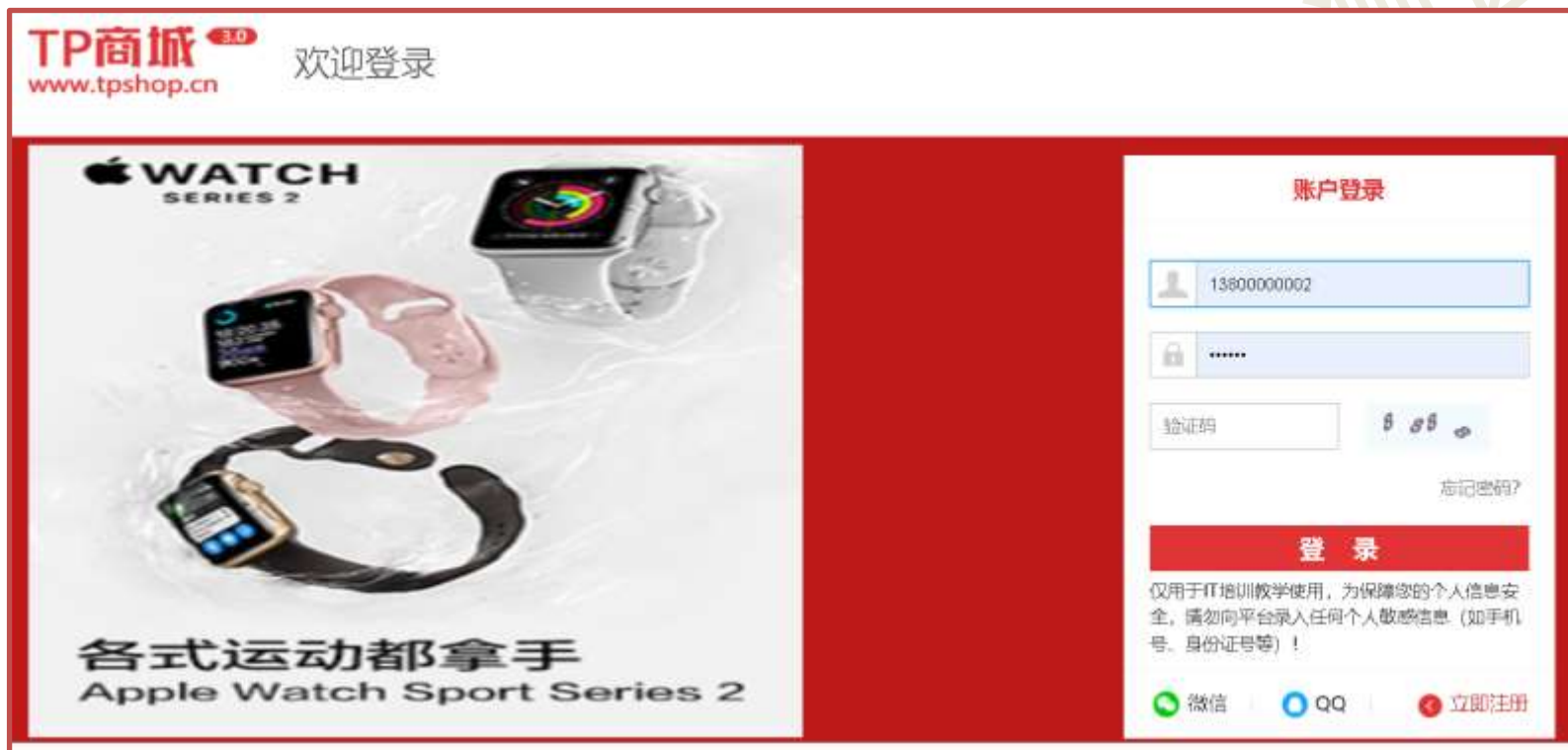
- **概念：** selenium利用选择器定位元素的定位方式。



案例

登录页面展示

需求：针对Tpshop前台商城系统登录页面的账号、密码、验证码输入框，登录按钮进行元素定位，并记录下定位的信息。



网页地址：<http://hmshop-test.itheima.net/Home/user/login.html>

总结

1. 为什么要进行元素定位

我们要通过查看元素的属性信息来帮助我们找到这个元素

2. 元素定位的主要方式有哪些

- ① css定位
- ② Xpath定位
- ③ class_name定位
- ④ id定位

05

TPshop登录

- 元素定位方法
- 模拟用户操作方法
- 元素强制等待方法
- 获取元素文本信息

案例

登录页面展示

需求：自动完成登录成功，并打印关键信息校验登录成功

TP商城 3.0 欢迎登录
www.tpshop.cn



网页地址: <http://hmshop-test.itheima.net/Home/user/login.html>

登录成功账号: 13800000001 密码: 123456

登录页面展示

- 元素定位方法
 - `webdriver.find_element(by=By.所要选择定位方式的类属性, value=所选择定位方式所要求的值)`
- 模拟元素点击
 - `element.click()`
- 模拟元素输入
 - `element.send_keys("输入的内容")`
- 等待元素出现再操作
 - `time.sleep(秒数)`

步骤

登录成功操作步骤

- ①导包
- ②创建浏览器驱动对象
- ③访问页面
- ④元素定位并模拟用户操作
- ⑤暂停几秒钟
- ⑥浏览器驱动对象退出

登录成功

```
# ①导包
import time
from selenium import webdriver
from selenium.webdriver.common.by import By

# ②创建浏览器驱动对象
driver = webdriver.Chrome()

# ③访问页面
driver.get("http://hmshop-test.itheima.net/Home/user/login.html")

# ④元素定位并模拟用户操作
# 定位用户名输入框，并模拟输入账号 使用id进行元素定位 模拟页面的输入操作
driver.find_element(By.ID, "username").send_keys("13800000001")
# 定位密码输入框，并模拟输入密码 使用Xpath进行元素定位
driver.find_element(By.XPATH, "//*[@type='password']").send_keys("123456")
# 定位验证码输入框，并模拟输入验证码 使用CSS进行元素定位
driver.find_element(By.CSS_SELECTOR, "#verify_code").send_keys("8888")
# 定位登录按钮，并模拟点击 使用Class进行元素定位 模拟页面的点击操作
driver.find_element(By.CLASS_NAME, "J-login-submit").click()

# ⑤暂停几秒钟
time.sleep(3)

# ⑥浏览器驱动对象退出
driver.quit()
```

获取登录成功信息

- 获取元素文本信息

获取元素文本

element.text

- 代码

```
time.sleep(2)
'''
    知识点：获取元素文本信息
'''
# 获取首页"安全退出"文本信息
msg = driver.find_element(By.XPATH, "//*[text()='安全退出']").text
print(msg)
# ⑤暂停几秒钟
time.sleep(3)
# ⑥浏览器驱动对象退出
driver.quit()
```

总结

1. 元素定位方法

`driver.find_element(by=By.所要选择定位方式的类属性, value=所选择定位方式所要求的值)`

2. 模拟用户点击

`element.click()`

3. 模拟用户输入

`element.send_keys(输入的文本信息)`

4. 等待元素出现

`time.sleep(秒数)`

5. 获取元素文本信息

`element.text`

06

新增商品成功

- 窗口最大化、Frame切换，Select下拉框操作
- 隐式等待、显示等待

案例

新增商品操作页面展示



登录页地址: <https://hmsshop-test.itheima.net/index.php/Admin/Admin/login>

账号: admin 密码: HM_2023_test

步骤

新增商品成功操作步骤

1、导包

2、创建浏览器驱动对象

3、打开Web页面

4、完成登录

5、进入商城列表

6、进入添加商品页面

7、填写并提交商品成功

8、暂停3秒

9、退出浏览器驱动

登录成功并切换到商品管理列表

```
# 导包
import time
from selenium.webdriver.common.by import By
from selenium import webdriver

# 创建浏览器驱动对象
driver = webdriver.Chrome()

# 打开登录页面
driver.get("http://hmshop-test.itheima.net/index.php/Admin/Admin/login")

# 完成登录
driver.find_element(By.CSS_SELECTOR, "[name='username']").send_keys("admin")
driver.find_element(By.CSS_SELECTOR, "[name='password']").send_keys("HM_2023_test")
driver.find_element(By.ID, "verify").send_keys("8888")
driver.find_element(By.CSS_SELECTOR, "[value='登录']").click()

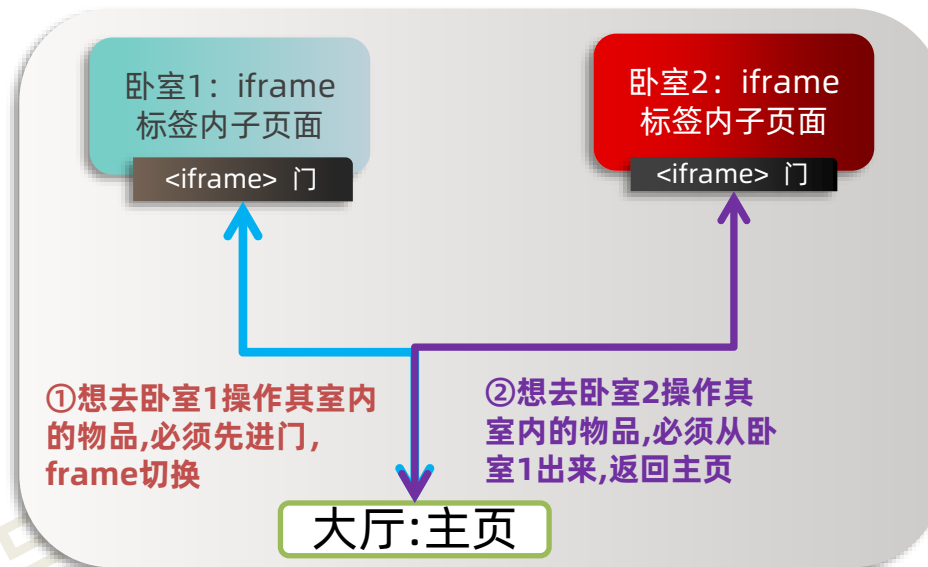
# 进入商城列表
driver.find_element(By.XPATH, "//li[@data-param='goods']").click()

# 暂停3秒
time.sleep(3)

# 退出浏览器驱动
driver.quit()
```


进入添加商品页

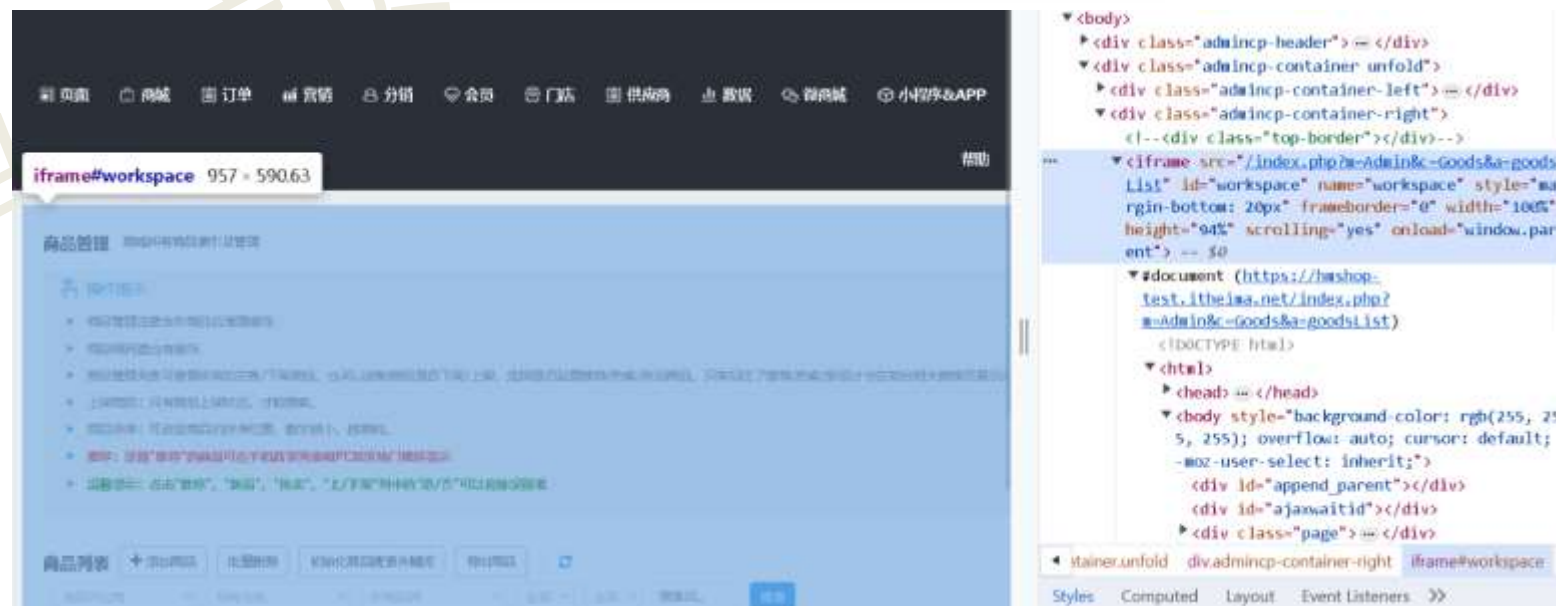
- 窗口最大化
 - webdriver.maximize_window()
- Frame切换
 - driver.switch_to.frame(frame_reference)
 - frame_reference: iframe标签元素对象



Frame

HTML页面中的一种框架，主要作用是在当前页面中指定区域显示另一页面元素。

- frameset形式
- iframe标签形式



进入添加商品页

- 窗口最大化

```
# 创建浏览器驱动对象
driver = webdriver.Chrome()
...
知识点：窗口最大化
...
driver.maximize_window()
# 打开登录页面
driver.get("http://hmshop-test.itheima.net/index.php/Admin/Admin/login")
```

- Frame切换，进入添加商品页

```
# 进入商城列表
driver.find_element(By.XPATH, "//li[@data-param='goods']").click()
...
知识点：Frame切换
...
driver.switch_to.frame(driver.find_element(By.XPATH, "//iframe[@id='workspace']"))
# 进入添加页面
driver.find_element(By.CSS_SELECTOR, "[href='/index.php/Admin/goods/addEditGoods']").click()
```

填写并提交新增商品信息

● Select下拉框

1. 导包

```
from selenium.webdriver.support.select import Select
```

2. 创建select对象

```
select = Select(element)
```

3. 选择选项

```
select.select_by_value(value) 根据选项value属性值
```

```
select.select_by_visible_text(text) 根据选项文本
```

注意:

Select类实现选项选择只适用于HTML原生态的<select>+<option>的下拉框。

* 商品分类

鞋、箱包、珠宝、手表

精品男包

商品简介

商品货号

如果不填会自动生成

SPU

SKU

商品品牌

所有品牌

<dd class="opt">

<select name="cat_id" id="cat_id" class="small form-control">

<option value="0">请选择商品分类</option>

<option value="7">鞋、箱包、珠宝、手表 </option>

<option value="8">运动户外 </option>

<option value="5">男装、女装、童装、内衣 </option>

<option value="6">个人化妆 </option>

<option value="11">图书、音像、电子书 </option>

<option value="2">家用电器 </option>

<option value="3">电脑、办公 </option>

<option value="4">家居、家具、家装、厨具 </option>

<option value="9">汽车用品 </option>

<option value="10">母婴用品、儿童玩具 </option>

</select>

填写并提交新增商品信息

- 核心代码：Select下拉框的处理，选择商品分类

```
# 下拉框操作
'''
    知识点: Select下拉框处理
'''

# 选择商品一级分类
Select(driver.find_element(By.CSS_SELECTOR, "[name='cat_id']")).select_by_value("6")

# 强调：选择下一级分类，必须先点击
driver.find_element(By.CSS_SELECTOR, "[name='cat_id_2']").click()
time.sleep(2) 点击之后，选择选项之前，需要等待元素的加载

# 选择商品二级分类
Select(driver.find_element(By.CSS_SELECTOR, "[name='cat_id_2']")).select_by_value("43")

# 点击商品三级分类
driver.find_element(By.CSS_SELECTOR, "[name='cat_id_3']").click()
time.sleep(2)

# 选择商品三级分类
Select(driver.find_element(By.CSS_SELECTOR, "[name='cat_id_3']")).select_by_visible_text("漱口水")
driver.find_element(By.CSS_SELECTOR, "[name='cat_id_2']").click()
```

代码优化：隐式等待和显示等待

在定位页面元素时，如果定位的元素还没有加载出现，会指定等待一段时间。

```
# 选择商品一级分类  
Select(driver.find_element(By.C
```

```
# 强调：选择下一级分类，必须先点击  
driver.find_element(By.CSS_SELECTOR, 'div[class="item"]>a')  
time.sleep(2)
```

```
# 选择商品二级分类  
Select(driver.find_element(By.C
```

```
# 点击商品三级分类  
driver.find_element(By.CSS_SELECTOR, 'div[class="item"]>a')  
time.sleep(2)
```

```
# 选择商品三级分类  
Select(driver.find_element(By.C  
driver.find_element(By.CSS_SELECTOR, 'div[class="item"]>a')
```

```
driver.find_element(By.CSS_SELECTOR, 'div[class="item"]>a')  
driver.find_element(By.CSS_SELECTOR, 'div[class="item"]>a')
```



- 网络速度慢
- 电脑配置低
- 服务器处理请求慢

解决方案

强制等待

time.sleep()影响运行效率

隐式等待

显示等待

代码优化：隐式等待和显示等待

- 隐式等待
 - 只用设置一次，对全局生效
 - `webdriver.implicitly_wait(等待的秒数)`
- 显示等待
 - 只对指定元素生效
 - 等待的元素对象 = `WebDriverWait(浏览器驱动对象, 等待的秒数).until(函数)`
 - 函数的第一个参数就是WebDriverWait实例对象传递的浏览器驱动对象
 - 可以采用函数简写形式--匿名函数：例如 `lambda driver : driver.find_element(By.ID, "someId")`

代码优化：隐式等待和显示等待

	隐式等待	显示等待
作用域	只用设置一次，对全局生效	只对指定元素生效
使用方法	<code>driver.implicitly_wait(timeout)</code>	通过WebDriverWait对象
抛出异常	<code>NoSuchElementException</code>	<code>TimeOutException</code>

- 隐式等待和显示等待都是智能等待，在一定时间范围内不断查找元素，一旦找到立刻结束查找，继续执行代码。
- 隐式等待会一直等待整个页面，脚本中需要定位元素加载完成，才会执行下一步。
- 显示等待允许你等待一个条件发生后再去定位该元素
- 其实隐式等待和显示等待在本质上是一致的，只是显示等待多了一个指定元素条件超时时间，在使用场景上，可以使用隐式等待来做一个全局的控制，例如设置全局隐式等待10秒；如果某个控件比较特殊，需要更长的时间加载，比如十几秒或者更长，就可以使用显示等待对其进行单独处理；

代码优化：隐式等待和显示等待

- 添加隐式等待

```
# 导包
import ...

# 创建浏览器对象
driver = webdriver.Chrome()

# 设置隐式等待
driver.implicitly_wait(30)

# 设置窗口最大化
driver.maximize_window()

# 访问Web页面
driver.get("http://hishop-test.itheima.net/index.php/Admin/Admin/login")
```

- 添加显示等待

```
# 完成登录
# 显示等待

WebDriverWait(driver, 30).until \
    (lambda x: driver.find_element(By.CSS_SELECTOR, "[name='username']").send_keys("admin"))

WebDriverWait(driver, 30).until \
    (lambda x: driver.find_element(By.CSS_SELECTOR, "[name='password']").send_keys("HM_2023_test"))

WebDriverWait(driver, 30).until \
    (lambda x: driver.find_element(By.ID, "verify").send_keys("8888"))

WebDriverWait(driver, 30).until \
    (lambda x: driver.find_element(By.CSS_SELECTOR, "[value='登录']").click())
```




总结

1. 如何实现浏览器窗口最大化

```
webdriver.maximize_window()
```

2. 如何实现Frame切换

```
driver.switch_to.frame(frame_reference)
```

3. Select下拉框操作

创建对象: `select= Select(element)`

选择选项: `select.select_by_value(value)`—根据选项value属性值

`select.select_by_visible_text(text)`—根据选项文本

4. 元素等待的实现方式

强制等待、隐式等待和显示等待

5. 隐式等待的实现方式

```
webdriver.implicitly_wait(timeout)
```

6. 显示等待的实现方式

```
WebDriverWait(driver, timeout).until(lambda x:  
x.find_element(元素定位方式, 定位值))
```

07

新增用户地址

- 鼠标悬停
- 定位一组元素
- 页面刷新

案例 新增用户地址操作页面展示

需求：自动完成新增用户地址，并校验完成之后已保存地址数量增加1。

账户登录

手机号/邮箱

密码

验证码

忘记密码?

登录

仅用于IT培训教学使用，为保障您的个人信息安全，请勿向平台录入任何个人敏感信息（如手机号、身份证等）！

微信 QQ 立即注册

TP商城 www.tpshop.cn

我的商城

个人中心

交易中心

我的订单

我的收货地址

资产中心

我的钱包

我的优惠券

我的收藏

我的足迹

个人中心

个人信息

账号绑定

地址管理

安全设置

账户设置

个人信息

收货地址

TPShop 升级商城

13800000001

永泰钻石

账户安全: 中低

我的订单

下单时间: 2024-01-11 23:49:40 订单编号: 20240111234

小米手机6,十多项黑科技,极速快充

我的收藏

地址管理

收货人	收货地址	联系电话	操作
地址	河北 保定市 南市区 西二门大街顺天街北一边	13548632158	默认地址 修改 删除
小强	北京 朝阳区 三环到四环之间 我就是不告诉你地址	13623212333	设为默认 修改 删除
小强	北京 朝阳区 三环到四环之间 我就是不告诉你地址	13623212333	设为默认 修改 删除
小强	北京 朝阳区 三环到四环之间 我就是不告诉你地址	13623212333	设为默认 修改 删除

已保存+条地址，还可以保存+条地址

新增收货地址

收货人: ①

手机号码或电话: (固定电话格式:xx-xx(例如: 0755-86140485)) ②

收货地址: 请选择 ③

详细地址: 详细地址 ④

邮编:

保存 ⑤ 取消

步骤

新增用户地址操作步骤

1.创建浏览器对象（窗口最大化、隐式等待）

2.访问登录页面

3.登录成功

4.进入新增用户地址页面

5.填写并提交新增用户地址信息

6.暂停几秒

7.退出浏览器驱动对象

进入地址管理页面

● 鼠标悬停

导包

```
from selenium.webdriver import ActionChains
```

实例化鼠标对象

```
action = ActionChains(driver)
```

调用鼠标方法

```
action.move_to_element(element) 鼠标悬停
```

执行鼠标操作

```
action.perform()
```

调用鼠标方法并不会去执行鼠标操作，必须调用perform才会执行

● 核心代码

知识点：鼠标悬停

练习：鼠标悬停之后，选择下拉框

```
'''  
知识点：鼠标悬停  
练习：鼠标悬停之后，选择下拉框  
'''  
# 创建鼠标对象  
action = ActionChains(driver)  
# 调用悬停方法  
action.move_to_element(driver.find_element(By.XPATH, "//div/span[text()='账户设置']"))  
# 执行鼠标操作指令  
action.perform()  
# 点击"收货地址"  
driver.find_element(By.XPATH, "//a[text()='收货地址']").click()
```

填写并提交新增地址

- 定位一组元素方法

- 元素列表 = webdriver.find_elements(by=By.所要选择定位方式的类属性, value=所选择定位方式所要求的值)

- 页面刷新

- driver.refresh()

业务说明：新增地址后，需刷新页面才能获取最新已保存地址数



填写并提交新增地址

- 获取已保存地址数量

```
'''
知识点：定位一组元素
需求：新增地址之后，校验地址信息+1
'''
# 获取当前用户的地址数量
old_num = driver.find_elements(By.XPATH, "//p/em[@class='red']")[0].text
print(old_num)
```

- 刷新页面

```
'''
知识点：页面的刷新
'''
driver.refresh()
# 刷新页面
# 获取新增之后的地址数量
new_num = driver.find_elements(By.XPATH, "//p/em[@class='red']")[0].text
print(new_num)
```

总结

1. 模拟鼠标悬停操作

实例化鼠标对象：`action = ActionChains(driver)`

调用鼠标方法(悬停)：`action.move_to_element(element)`

执行鼠标操作：`action.perform()`

2. 定位一组元素方法

元素列表 = `webdriver.find_elements(By.定位方式, "value")`

3. 模拟页面刷新

`driver.refresh()`